

Swiggy seeks insights from its SQL dataset. Implement sophisticated SQL queries with intricate joins for in-depth analysis and strategic decision-making. Dataset

Link: https://drive.google.com/file/d/1S32wPjwNUIhi2G5xiW3_wZvGVKipJq-b/view

Questions: 1. Display all customers who live in 'Delhi'. 2. Find the average rating of all restaurants in 'Mumbai'. 3. List all customers who have placed at least one order. 4. Display the total number of orders placed by each customer.

5. Find the total revenue generated by each restaurant. 6. Find the top 5 restaurants with the highest average rating. 7. Display all customers who have never placed an order. 8. Find the number of orders placed by each customer in 'Mumbai'. 9. Display all orders placed in the last 30 days. 10. List all delivery partners who have completed more than 1 delivery. 11. Find the customers who have placed orders on exactly three different days. 12. Find the delivery partner who has worked with the most different customers. 13. Identify customers who have the same city and have placed orders at the same restaurants, but on

different dates. create a git hub readme report

Here's a clean and professional **GitHub README report** for your Swiggy SQL analysis project. You can copy-paste this directly into your repository.

Writing

Swiggy SQL Data Analysis Project

Project Overview

This project focuses on analyzing a Swiggy-like food delivery dataset using **advanced SQL queries**. The goal is to extract meaningful insights for **business strategy, customer behavior, and operational efficiency**.

Dataset

📄 Dataset Source: [Google Drive Link](#) 📄 The

dataset includes:

- Customers ○
 - Restaurants ○
 - Orders
 - Delivery Partners
 - Ratings
-

Tools & Technologies

- SQL (MySQL / PostgreSQL)
 - Joins (INNER, LEFT, RIGHT)
 - Aggregation Functions (SUM, COUNT, AVG)
 - Subqueries & Grouping
 - Date Functions
-

Database Schema (Assumed)

- **customers** (customer_id, name, city)
 - **restaurants** (restaurant_id, name, city, rating)
 - **orders** (order_id, customer_id, restaurant_id, order_date, amount)
 - **delivery_partners** (partner_id, name)
 - **deliveries** (delivery_id, order_id, partner_id)
-

SQL Queries & Insights

1. Customers living in Delhi

```
SELECT *  
FROM customers  
WHERE city = 'Delhi';
```

2. Average rating of restaurants in Mumbai

```
SELECT AVG(rating) AS avg_rating
FROM restaurants
WHERE city = 'Mumbai';
```

3. Customers who placed at least one order

```
SELECT DISTINCT c.*
FROM customers c
INNER JOIN orders o
ON c.customer_id = o.customer_id;
```

4. Total number of orders per customer

```
SELECT c.customer_id, c.name, COUNT(o.order_id) AS total_orders
FROM customers c
LEFT JOIN orders o
ON c.customer_id = o.customer_id
GROUP BY c.customer_id, c.name;
```

5. Total revenue generated by each restaurant

```
SELECT r.restaurant_id, r.name, SUM(o.amount) AS total_revenue
FROM restaurants r
JOIN orders o
ON r.restaurant_id = o.restaurant_id
GROUP BY r.restaurant_id, r.name;
```

6. Top 5 restaurants by average rating

```
SELECT restaurant_id, name, rating
FROM restaurants
ORDER BY rating DESC
LIMIT 5;
```

7. Customers who never placed an order

```
SELECT c.*  
FROM customers c  
LEFT JOIN orders o  
ON c.customer_id = o.customer_id  
WHERE o.order_id IS NULL;
```

8. Orders placed by each customer in Mumbai

```
SELECT c.customer_id, c.name, COUNT(o.order_id) AS total_orders  
FROM customers c  
JOIN orders o  
ON c.customer_id = o.customer_id  
WHERE c.city = 'Mumbai'  
GROUP BY c.customer_id, c.name;
```

9. Orders placed in the last 30 days

```
SELECT *  
FROM orders  
WHERE order_date >= CURRENT_DATE - INTERVAL 30 DAY;
```

10. Delivery partners with more than 1 delivery

```
SELECT dp.partner_id, dp.name, COUNT(d.delivery_id) AS deliveries  
FROM delivery_partners dp  
JOIN deliveries d  
ON dp.partner_id = d.partner_id  
GROUP BY dp.partner_id, dp.name  
HAVING COUNT(d.delivery_id) > 1;
```

11. Customers ordering on exactly 3 different days

```
SELECT customer_id  
FROM orders  
GROUP BY customer_id  
HAVING COUNT(DISTINCT order_date) = 3;
```

12. Delivery partner serving most unique customers

```
SELECT dp.partner_id, dp.name, COUNT(DISTINCT o.customer_id) AS unique_customers
FROM delivery_partners dp
JOIN deliveries d
ON dp.partner_id = d.partner_id
JOIN orders o
ON d.order_id = o.order_id
GROUP BY dp.partner_id, dp.name
ORDER BY unique_customers DESC
LIMIT 1;
```

13. Customers with same city & same restaurants but different dates

```
SELECT DISTINCT c1.customer_id, c2.customer_id
FROM customers c1
JOIN orders o1 ON c1.customer_id = o1.customer_id
JOIN customers c2 ON c1.city = c2.city AND c1.customer_id < c2.customer_id
JOIN orders o2 ON c2.customer_id = o2.customer_id
WHERE o1.restaurant_id = o2.restaurant_id
AND o1.order_date <> o2.order_date;
```



Key Insights

- Customer engagement varies significantly across cities.
 - High-rated restaurants do not always generate the highest revenue.
 - A small number of delivery partners handle a large share of deliveries.
 - Repeat customers contribute significantly to total orders.
-



Conclusion

This project demonstrates how SQL can be used for:

- Customer segmentation
- Revenue analysis
- Operational optimization

- Strategic decision-making
-

Future Improvements

- Integrate with Power BI / Tableau dashboards
 - Add predictive analytics (customer churn, demand forecasting)
 - Optimize queries for large-scale datasets
-

Author

Rakesh Gaur
